

Graph Representation of Symbolic Execution

1 Symbolic Execution

Symbolic execution is a static code analysis technique that considers all possible input values under which each part of a program can be executed. In this way, the behavior of the program is determined for execution paths corresponding to the analyzed inputs. Symbolic execution can be used for various code analysis tasks, such as vulnerability and bug detection, maximizing test coverage, verifying whether a specific instruction can be executed, and others.

In this section, symbolic execution is described based on its implementation in VSHARP. To better understand the concept, let us consider a simple example shown in Listing 1.

1.1 Example Program

```
1 def function(x: int):
2     if x < 0:
3         y = x + 1
4         return y
5     else:
6         s = other_function(x)
7         s += 1
8         return s
9
10 def other_function(x: int):
11     if x == -5:
12         return 0
13     else:
14         return x
```

Listing 1: Example program

At each step of program analysis, the configuration of the symbolic machine can be represented as a graph. The method under test is divided into basic blocks, and a control-flow graph is constructed from them. Additional vertices are added to represent possible symbolic execution states.

These states are responsible for exploring the control-flow graph through their movement. Each state is characterized by a logical formula called a *path condition*. This formula defines the constraints under which execution can reach a specific basic block of code.

Formally, the resulting graph can be described as:

$G = (V, E)$, where $V = V_1 \cup V_2 \cup V_3$ and $E = E_{V_1} \cup E_{in} \cup E_{parent_of} \cup E_{history} \cup E_{call} \cup E_{return} \cup E_{pc_root} \cup E_{V_3}$, where:

- V_1, E_{V_1} — sets of vertices and edges of the intraprocedural control-flow graph;
- V_2 — symbolic states;
- V_3, E_{V_3} — sets of vertices and edges of the graph constructed from the path condition;
- $E_{in} = \{(u, v) \mid (u \in V_2, v \in V_1)\}$, where state u is located in basic block v ;
- $E_{parent_of} = \{(u, v) \mid (u, v \in V_2)\}$, where state u generated state v ;
- $E_{history} = \{(u, v) \mid (u \in V_2, v \in V_1)\}$, where state u visited basic block v ;
- $E_{call} = \{(u, v) \mid (u, v \in V_1)\}$, where basic block u ends with a method call whose entry block is v ;
- $E_{return} = \{(u, v) \mid (u, v \in V_1)\}$, where block u ends with a return instruction and control is transferred to block v ;
- $E_{pc_root} = \{(u, v) \mid u \in V_2, v \in V_3\}$, where vertex v is the root of the logical formula characterizing state u .

Thus, the entire graph consists of intraprocedural subgraphs connected by E_{call} and E_{return} edges, a tree of symbolic states constructed during symbolic execution, and logical formulas represented as graphs.

The $E_{history}$ and E_{in} edges connect control-flow graph vertices with symbolic states. The E_{pc_root} edges define the connection between symbolic states and the root vertex of the path condition.

1.2 Vertex Attributes

Vertices and edges of several types contain attributes.

1.2.1 Attributes of V_1 Vertices

All vertices $v \in V_1$ contain the following attributes:

- **Id** — unique identifier of the basic block;
- **InCoverageZone** — whether the vertex belongs to the target coverage zone;
- **BasicBlockSize** — size of the basic block in instructions;
- **CoveredByTest** — whether a test covering this block has already been generated;
- **VisitedByState** — whether the block has been visited by any symbolic state;
- **TouchedByState** — whether any symbolic state has ever reached this block;
- **ContainsCall** — whether the block contains a function call;
- **ContainsThrow** — whether the block contains a throw instruction.

1.2.2 Attributes of V_2 Vertices

All vertices $v \in V_2$ contain the following attributes:

- **Id** — unique identifier of the symbolic state;
- **PathConditionSize** — size of the path condition;
- **VisitedAgainVertices** — total number of revisited vertices;
- **VisitedNotCoveredVerticesInZone** — number of uncovered vertices visited inside the target zone;
- **VisitedNotCoveredVerticesOutOfZone** — number of uncovered vertices visited outside the target zone;
- **StepWhenMovedLastTime** — execution step when the state was last moved;
- **InstructionsVisitedInCurrentBlock** — number of instructions visited in the current block.

1.2.3 Attributes of $E_{history}$ Edges

All edges $(u, v) \in E_{history}$ contain:

- **NumOfVisits** — number of visits of vertex v by state u ;
- **StepWhenVisitedLastTime** — last step when state u visited vertex v .

1.2.4 Attributes of Path Condition Vertices

Vertices of the logical formula graph $v \in V_3$ contain:

- **Id** — unique vertex identifier;
- **Type** — type of logical term.

1.3 Configuration Example

The graph described above together with all attributes is referred to as the graph representation of the symbolic machine configuration.

Suppose that after several execution steps, two symbolic states are available. One of them can be selected and moved further through the control-flow graph.

Assume that the second state is selected. Since the previous vertex contains a call to `other_function`, the control-flow graph of the called method is constructed and connected to the existing graph using call and return edges.

After another execution step, suppose the symbolic state reaches the condition $x == -5$.

The updated path condition becomes $x \neq -5 \wedge x \geq 0$, which is represented as a graph.

It is important to note that identical terms are represented by shared vertices rather than duplicated ones. For example, the variable x may appear multiple times in the formula graph while corresponding to only a single graph vertex.

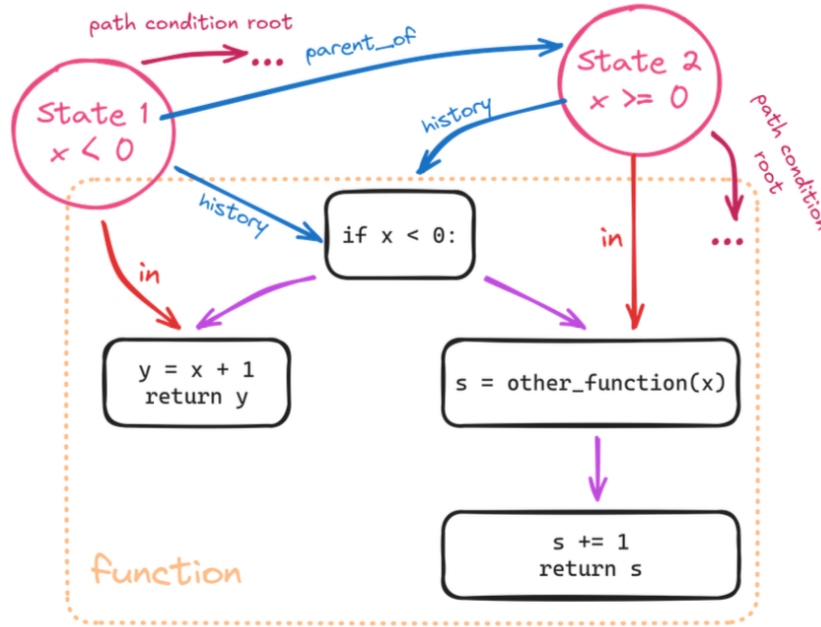


Figure 1: Configuration of the symbolic machine during execution of the example program

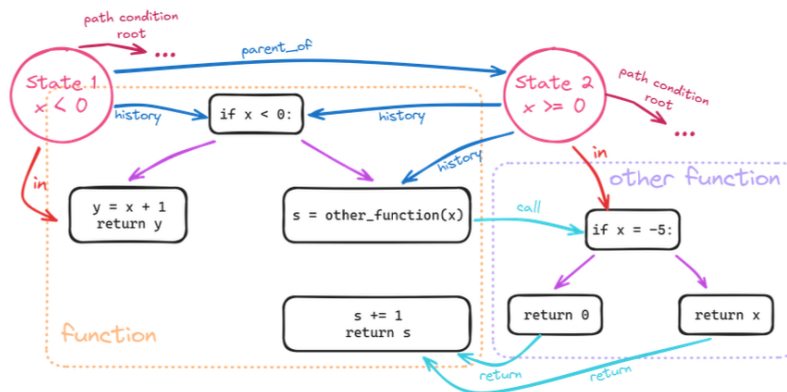


Figure 2: New symbolic machine configuration after moving State 2

If a generated condition becomes unsatisfiable, the corresponding execution branch is discarded. Formula satisfiability is determined using an SMT solver.

Source Thesis:

https://se.math.spbu.ru/thesis_download?thesis_id=1588

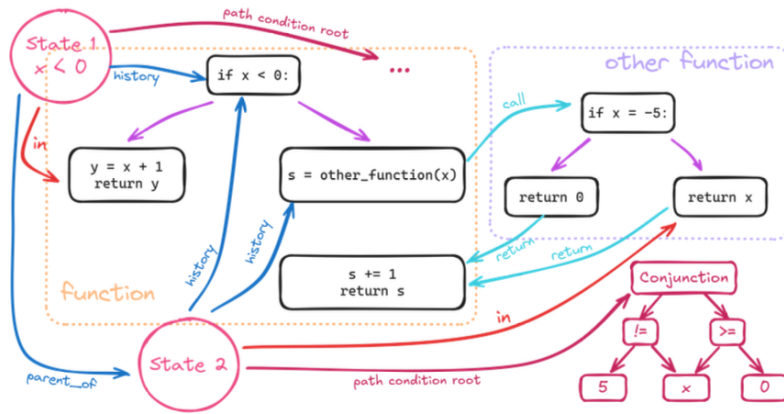


Figure 3: Configuration with updated path condition